

Banking Automation Framework - Source Code Documentation

Project: Banking Automation Framework (Guru99)

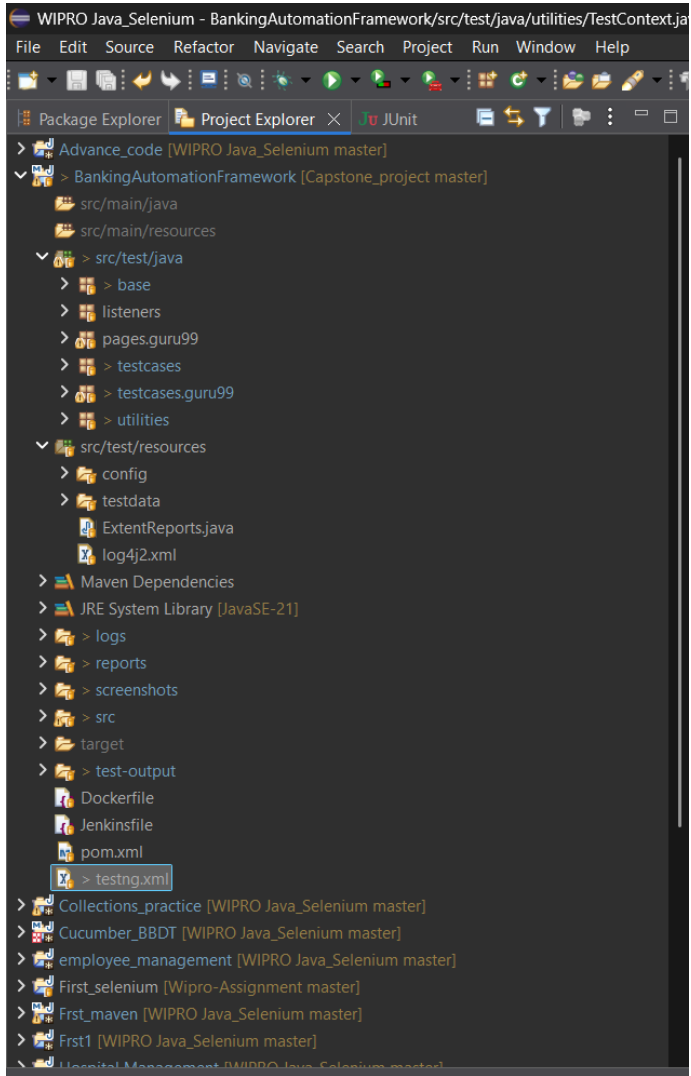
Technology Used:

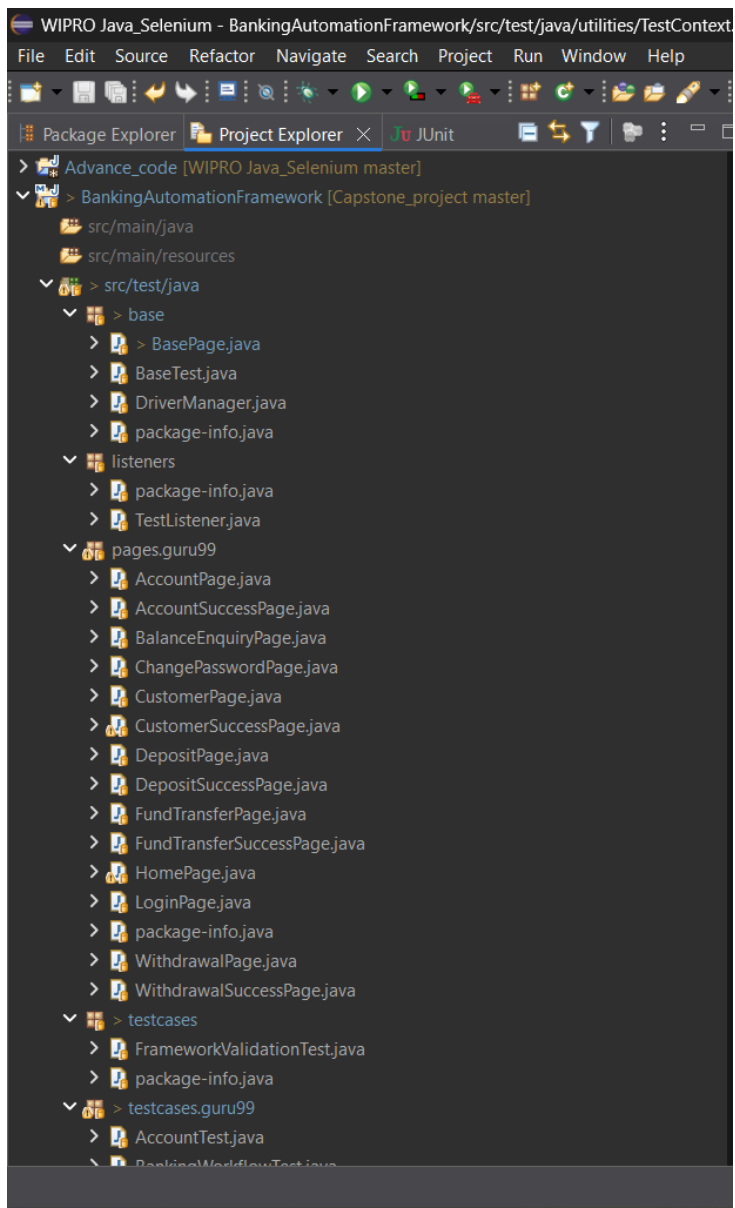
- Selenium
- Java
- TestNG
- Maven
- Jenkins
- Docker
- Extent Reports

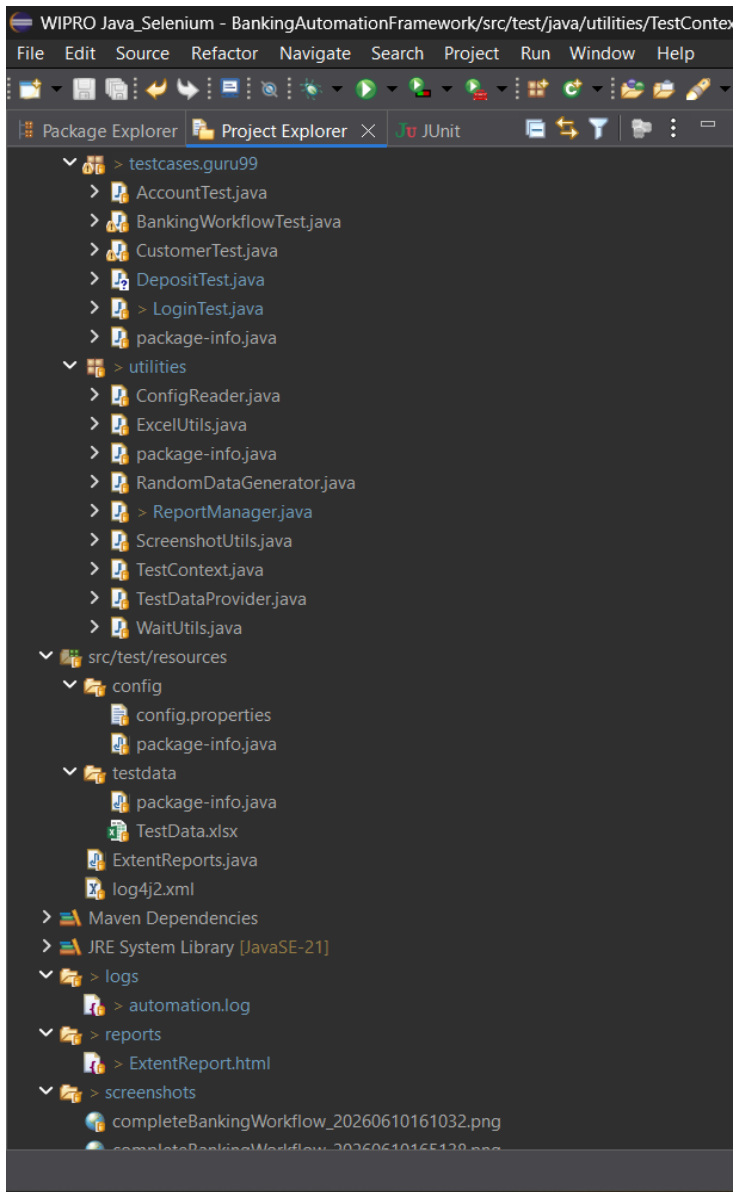
Project Overview

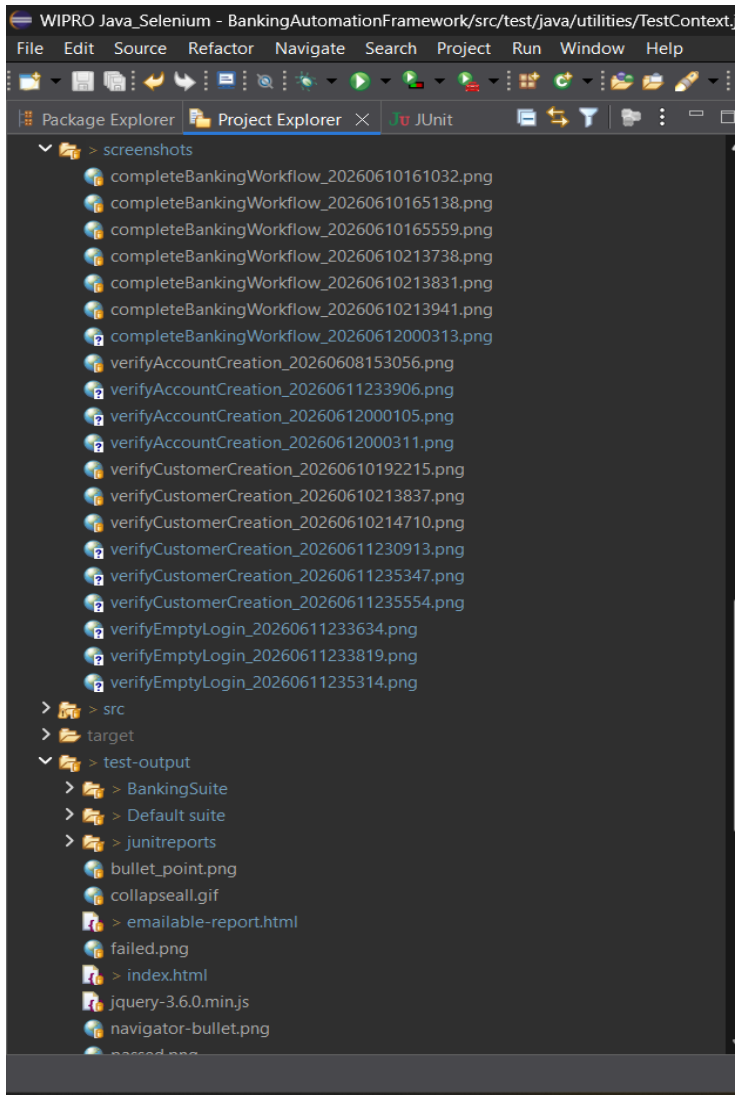
Framework developed using Page Object Model (POM) for Guru99 Banking Application automation.

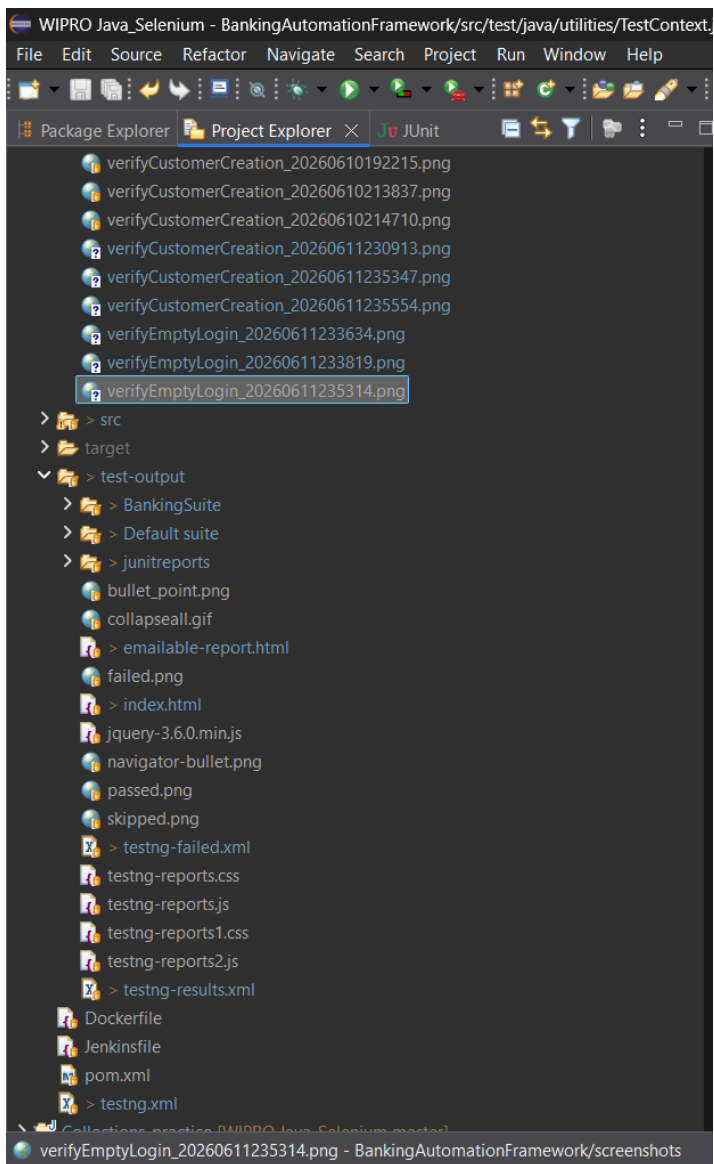
Project Structure











Base Package

BasePage:

package base;

import org.openqa.selenium.WebDriver;

```
import org.openqa.selenium.support.PageFactory;
```

```
public class BasePage {
```

```
    protected WebDriver driver;
```

```
    public BasePage(WebDriver driver) {
```

```
        this.driver = driver;
```

```
        PageFactory.initElements(driver,this);
```

```
    }
```

```
}
```

BaseTest:

```
package base;
```

```
import org.testng.annotations.AfterMethod;
```

```
import org.testng.annotations.BeforeMethod;
```

```
import utilities.ConfigReader;
```

```
import utilities.TestContext;
```

```
public class BaseTest {
```

```
    @BeforeMethod
```

```
    public void setup() {
```

```

        DriverManager.initializeDriver(
            ConfigReader.getProperty("browser"));

        DriverManager.getDriver().get(
            ConfigReader.getProperty("url"));
    }

    @AfterMethod
    public void tearDown() {

        DriverManager.quitDriver();

        TestContext.setCustomerId(null);
        TestContext.setAccountId(null);
    }
}

```

DriverManager:

```

package base;

import io.github.bonigarcia.wdm.WebDriverManager;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.edge.EdgeDriver;
import org.openqa.selenium.firefox.FirefoxDriver;

```



```
import org.openqa.selenium.chrome.ChromeOptions;

public class DriverManager {

    private static final ThreadLocal<WebDriver> driver =
        new ThreadLocal<>();

    public static void initializeDriver(String browser) {

        switch (browser.toLowerCase()) {

            case "chrome":

                ChromeOptions options =
                    new ChromeOptions();

                if(System.getProperty("os.name")
                    .toLowerCase()
                    .contains("linux")) {

                    options.addArguments("--headless=new");
                    options.addArguments("--no-sandbox");
                    options.addArguments("--disable-dev-shm-usage");
                    options.addArguments("--disable-gpu");
                    options.addArguments("--window-size=1920,1080");
                }
            }
        }
    }
}
```

```
        driver.set(new ChromeDriver(options));
        break;

    case "edge":
        WebDriverManager.edgedriver().setup();
        driver.set(new EdgeDriver());
        break;

    case "firefox":
        WebDriverManager.firefoxdriver().setup();
        driver.set(new FirefoxDriver());
        break;

    default:
        throw new RuntimeException(
            "Unsupported Browser");
    }

    getDriver().manage().window().maximize();
}

public static WebDriver getDriver() {

    return driver.get();
}
```

```
public static void quitDriver() {  
  
    if(driver.get()!=null) {  
  
        driver.get().quit();  
  
        driver.remove();  
    }  
}  
}
```

listeners Package

TestListeners

```
package listeners;  
  
import com.aventstack.extentreports.*;  
import org.testng.*;  
import utilities.ReportManager;  
import utilities.ScreenshotUtils;  
  
public class TestListener implements ITestListener {  
  
    private static ExtentReports extent =  
        ReportManager.getReportInstance();
```

```
private static ThreadLocal<ExtentTest> test =  
    new ThreadLocal<>();
```

```
@Override
```

```
public void onTestStart(ITestResult result) {
```

```
    System.out.println("Listener Triggered");
```

```
    test.set(  
        extent.createTest(  
            result.getMethod().getMethodName()));  
}
```

```
@Override
```

```
public void onTestSuccess(  
    ITestResult result) {
```

```
    test.get().pass("Test Passed");  
}
```

```
@Override
```

```
public void onTestFailure(  
    ITestResult result) {
```

```
System.out.println(  
    "Failure Listener Triggered");
```

```
String screenshotPath =  
    ScreenshotUtils.capture(  
        result.getMethod().getMethodName());
```

```
System.out.println(  
    "Screenshot Path = "  
        + screenshotPath);
```

```
test.get().fail(result.getThrowable());
```

```
test.get().addScreenCaptureFromPath(  
    screenshotPath);
```

```
}
```

```
@Override
```

```
public void onFinish(  
    ITestContext context) {
```

```
    extent.flush();
```

```
}
```

```
}
```

Pages.guru99 Package (POM)

AccountPage

```
package pages.guru99;

import base.BasePage;
import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.FindBy;
import org.openqa.selenium.support.ui.Select;
import utilities.WaitUtils;

public class AccountPage extends BasePage {

    private static final Logger logger =
        LogManager.getLogger(AccountPage.class);

    public AccountPage(WebDriver driver) {
        super(driver);
    }

    @FindBy(name = "cusid")
```

```
private WebElement customerIdTxt;
```

```
@FindBy(name = "selaccount")
```

```
private WebElement accountTypeDropdown;
```

```
@FindBy(name = "inideposit")
```

```
private WebElement initialDepositTxt;
```

```
@FindBy(name = "button2")
```

```
private WebElement submitBtn;
```

```
public void createAccount(
```

```
    String customerId,
```

```
    String accountType,
```

```
    String initialDeposit) {
```

```
    logger.info("Creating New Account");
```

```
    customerIdTxt.clear();
```

```
    customerIdTxt.sendKeys(customerId);
```

```
    Select select =
```

```
        new Select(accountTypeDropdown);
```

```
    select.selectByVisibleText(accountType);
```

```
initialDepositTxt.clear();  
initialDepositTxt.sendKeys(initialDeposit);  
  
System.out.println(  
    "Customer ID = "  
        + customerIdTxt.getAttribute("value"));
```

```
System.out.println(  
    "Initial Deposit = "  
        + initialDepositTxt.getAttribute("value"));
```

```
WaitUtils.waitForClickable(submitBtn);
```

```
    submitBtn.click();  
}  
}
```

AccountSuccessPage:

```
package pages.guru99;
```

```
import base.BasePage;  
import org.openqa.selenium.By;  
import org.openqa.selenium.WebDriver;  
import utilities.TestContext;
```



```

public class AccountSuccessPage extends BasePage {

    public AccountSuccessPage(WebDriver driver) {
        super(driver);
    }

    public String getAccountId() {

        String accountId =
            driver.findElement(
                By.xpath(
                    "//td[text()='Account ID']/following-sibling::td"))
                .getText();

        TestContext.setAccountId(accountId);

        return accountId;
    }
}

```

BalanceEnquiryPage

```

package pages.guru99;

```

```

import base.BasePage;

```

```
import org.apache.logging.log4j.LogManager;

import org.apache.logging.log4j.Logger;

import org.openqa.selenium.WebDriver;

import org.openqa.selenium.WebElement;

import org.openqa.selenium.support.FindBy;

import utilities.WaitUtils;


public class BalanceEnquiryPage extends BasePage {

    private static final Logger logger =

        LogManager.getLogger(BalanceEnquiryPage.class);


    public BalanceEnquiryPage(WebDriver driver) {

        super(driver);

    }


    @FindBy(name = "accountno")

    private WebElement accountNumberTxt;


    @FindBy(name = "AccSubmit")

    private WebElement submitBtn;


    public void searchAccount(String accountNumber) {

        logger.info("Performing Balance Enquiry");
```

```
accountNumberTxt.clear();  
accountNumberTxt.sendKeys(accountNumber);  
  
WaitUtils.waitForClickable(submitBtn);  
  
submitBtn.click();  
}  
}
```

CustomerPage

```
package pages.guru99;  
  
import base.BasePage;  
import org.apache.logging.log4j.LogManager;  
import org.apache.logging.log4j.Logger;  
import org.openqa.selenium.WebDriver;  
import org.openqa.selenium.WebElement;  
import org.openqa.selenium.support.FindBy;  
import utilities.WaitUtils;  
import org.openqa.selenium.Keys;  
  
public class CustomerPage extends BasePage {  
  
    private static final Logger logger =
```

```
LogManager.getLogger(CustomerPage.class);
```

```
public CustomerPage(WebDriver driver) {  
    super(driver);  
}
```

```
@FindBy(name = "name")
```

```
private WebElement customerNameTxt;
```

```
@FindBy(xpath = "//input[@value='m']")
```

```
private WebElement maleRadioBtn;
```

```
@FindBy(id = "dob")
```

```
private WebElement dobTxt;
```

```
@FindBy(name = "addr")
```

```
private WebElement addressTxt;
```

```
@FindBy(name = "city")
```

```
private WebElement cityTxt;
```

```
@FindBy(name = "state")
```

```
private WebElement stateTxt;
```

```
@FindBy(name = "pinno")
```

```
private WebElement pinTxt;
```

```
@FindBy(name = "telephoneno")
```

```
private WebElement mobileTxt;
```

```
@FindBy(name = "emailid")
```

```
private WebElement emailTxt;
```

```
@FindBy(name = "password")
```

```
private WebElement passwordTxt;
```

```
@FindBy(name = "sub")
```

```
private WebElement submitBtn;
```

```
public void createCustomer(  
    String name,  
    String dob,  
    String address,  
    String city,  
    String state,  
    String pin,  
    String mobile,  
    String email,  
    String password) {
```

```
logger.info("Creating Customer");
```

```
WaitUtils.waitForVisibility(customerNameTxt);
```

```
customerNameTxt.sendKeys(name);
```

```
System.out.println("Name Entered");
```

```
maleRadioBtn.click();
```

```
System.out.println("Gender Selected");
```

```
dobTxt.sendKeys(Keys.CONTROL + "a");
```

```
dobTxt.sendKeys(Keys.DELETE);
```

```
dobTxt.sendKeys("01011998");
```

```
System.out.println("DOB Entered");
```

```
addressTxt.sendKeys(address);
```

```
System.out.println("Address Entered");
```

```
cityTxt.sendKeys(city);
```

```
System.out.println("City Entered");
```

```
stateTxt.sendKeys(state);
```

```
System.out.println("State Entered");
```

```
pinTxt.sendKeys(pin);
```

```
System.out.println("PIN Entered");

mobileTxt.sendKeys(mobile);

System.out.println("Mobile Entered");

emailTxt.sendKeys(email);

System.out.println("Email Entered");

passwordTxt.sendKeys(password);

System.out.println("Password Entered");

WaitUtils.waitForClickable(submitBtn);

System.out.println("Name = " + customerNameTxt.getAttribute("value"));
System.out.println("DOB = " + dobTxt.getAttribute("value"));
System.out.println("Address = " + addressTxt.getAttribute("value"));
System.out.println("City = " + cityTxt.getAttribute("value"));
System.out.println("State = " + stateTxt.getAttribute("value"));
System.out.println("PIN = " + pinTxt.getAttribute("value"));
System.out.println("Mobile = " + mobileTxt.getAttribute("value"));
System.out.println("Email = " + emailTxt.getAttribute("value"));
System.out.println("Password = " + passwordTxt.getAttribute("value"));

submitBtn.click();
}
```

```
}
```

CustomerSuccessPage

```
package pages.guru99;
```

```
import base.BasePage;
```

```
import org.openqa.selenium.By;
```

```
import org.openqa.selenium.WebDriver;
```

```
import utilities.TestContext;
```

```
import java.time.Duration;
```

```
import org.openqa.selenium.By;
```

```
import org.openqa.selenium.WebElement;
```

```
import org.openqa.selenium.support.ui.ExpectedConditions;
```

```
import org.openqa.selenium.support.ui.WebDriverWait;
```

```
public class CustomerSuccessPage extends BasePage {
```

```
    public CustomerSuccessPage(WebDriver driver) {
```

```
        super(driver);
```

```
    }
```

```
    public String getCustomerId() {
```

```
        WebDriverWait wait =
```

```
            new WebDriverWait(
```

```
                driver,
```



```
Duration.ofSeconds(20));
```

```
WebElement customerElement =
```

```
    wait.until(
```

```
        ExpectedConditions
```

```
            .visibilityOfElementLocated(
```

```
                By.xpath(
```

```
                    "//td[text()='Customer ID']/following-sibling::td"))));
```

```
String customerId = customerElement.getText();
```

```
System.out.println(
```

```
    "Retrieved Customer ID = "
```

```
    + customerId);
```

```
TestContext.setCustomerId(customerId);
```

```
    return customerId;
```

```
}
```

```
}
```

DepositPage

```
package pages.guru99;
```

```
import base.BasePage;
```

```
import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.FindBy;
import utilities.WaitUtils;

public class DepositPage extends BasePage {

    private static final Logger logger =
        LogManager.getLogger(DepositPage.class);

    public DepositPage(WebDriver driver) {
        super(driver);
    }

    @FindBy(name = "accountno")
    private WebElement accountNumberTxt;

    @FindBy(name = "ammount")
    private WebElement amountTxt;

    @FindBy(name = "desc")
    private WebElement descriptionTxt;
```

```
@FindBy(name = "AccSubmit")
private WebElement submitBtn;

public void depositAmount(
    String accountId,
    String amount,
    String description) {

    logger.info("Performing Deposit");

    accountNumberTxt.sendKeys(accountId);

    amountTxt.sendKeys(amount);

    descriptionTxt.sendKeys(description);

    WaitUtils.waitForClickable(submitBtn);

    submitBtn.click();
}
}
```

DepositSuccessPage

```
package pages.guru99;
```

```
import base.BasePage;

import org.openqa.selenium.By;

import org.openqa.selenium.WebDriver;


public class DepositSuccessPage extends BasePage {


    public DepositSuccessPage(WebDriver driver) {
        super(driver);
    }


    public boolean isDepositSuccessful() {

        return driver.findElement(
            By.xpath("//*[contains(text(),'Transaction details of Deposit')]"))
            .isDisplayed();
    }


    public String getTransactionId() {

        try {

            return driver.findElement(
                By.xpath("//td[text()='Transaction ID']/following-sibling::td"))
                .getText();
        }
    }
}
```

```

    } catch (Exception e) {

        return "NOT AVAILABLE";
    }
}

public void printDepositDetails() {

    System.out.println(
        "Deposit Successful");

    System.out.println(
        "Transaction ID : "
        + getTransactionId());
}
}

```

FundTransferPage

```

package pages.guru99;

import base.BasePage;
import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;

```

```
import org.openqa.selenium.support.FindBy;

import utilities.WaitUtils;


public class FundTransferPage extends BasePage {


    private static final Logger logger =

        LogManager.getLogger(FundTransferPage.class);


    public FundTransferPage(WebDriver driver) {

        super(driver);

    }


    @FindBy(name = "payersaccount")
    private WebElement payerAccountTxt;


    @FindBy(name = "payeeaccount")
    private WebElement payeeAccountTxt;


    @FindBy(name = "ammount")
    private WebElement amountTxt;


    @FindBy(name = "desc")
    private WebElement descriptionTxt;


    @FindBy(name = "AccSubmit")
```

```
private WebElement submitBtn;

public void transferFund(
    String payerAccount,
    String payeeAccount,
    String amount,
    String description) {

    logger.info("Performing Fund Transfer");

    payerAccountTxt.sendKeys(payerAccount);
    payeeAccountTxt.sendKeys(payeeAccount);
    amountTxt.sendKeys(amount);
    descriptionTxt.sendKeys(description);

    WaitUtils.waitForClickable(submitBtn);
    submitBtn.click();
}
}
```

FundTransferSuccessPage

```
package pages.guru99;

import base.BasePage;
import org.openqa.selenium.By;
```

```

import org.openqa.selenium.WebDriver;

public class FundTransferSuccessPage extends BasePage {

    public FundTransferSuccessPage(WebDriver driver) {
        super(driver);
    }

    public boolean isFundTransferSuccessful() {

        return driver.findElement(
            By.xpath("//*[contains(text(),'Fund Transfer Details')]"))
            .isDisplayed();
    }

    public void printFundTransferDetails() {

        System.out.println(
            "Fund Transfer Successful");
    }
}

```

HomePage

```

package pages.guru99;

```



```
import base.BasePage;
```

```
import utilities.WaitUtils;
```

```
import java.time.Duration;
```

```
import org.apache.logging.log4j.LogManager;
```

```
import org.apache.logging.log4j.Logger;
```

```
import org.openqa.selenium.JavascriptExecutor;
```

```
import org.openqa.selenium.WebDriver;
```

```
import org.openqa.selenium.WebElement;
```

```
import org.openqa.selenium.support.FindingBy;
```

```
import org.openqa.selenium.support.ui.ExpectedConditions;
```

```
import org.openqa.selenium.support.ui.WebDriverWait;
```

```
public class HomePage extends BasePage {
```

```
    private static final Logger logger =
```

```
        LogManager.getLogger(HomePage.class);
```

```
    public HomePage(WebDriver driver) {
```

```
        super(driver);
```

```
    }
```

```
    @FindBy(xpath =
```

```
        "//td[contains(text(),'Manger Id') or contains(text(),'Manager Id')]")
```

```
private WebElement managerIdLabel;
```

```
@FindBy(linkText = "New Customer")
```

```
private WebElement newCustomerMenu;
```

```
@FindBy(linkText = "New Account")
```

```
private WebElement newAccountMenu;
```

```
@FindBy(linkText = "Deposit")
```

```
private WebElement depositMenu;
```

```
@FindBy(linkText = "Withdrawal")
```

```
private WebElement withdrawalMenu;
```

```
@FindBy(linkText = "Fund Transfer")
```

```
private WebElement fundTransferMenu;
```

```
@FindBy(linkText = "Change Password")
```

```
private WebElement changePasswordMenu;
```

```
@FindBy(linkText = "Balance Enquiry")
```

```
private WebElement balanceEnquiryMenu;
```

```
@FindBy(linkText = "Mini Statement")
```

```
private WebElement miniStatementMenu;
```

```
@FindBy(linkText = "Customised Statement")  
private WebElement customisedStatementMenu;
```

```
@FindBy(linkText = "Log out")  
private WebElement logoutMenu;
```

```
public boolean isManagerHomePageDisplayed() {
```

```
    WaitUtils.waitForVisibility(managerIdLabel);
```

```
    return managerIdLabel.isDisplayed();  
}
```

```
public void clickNewCustomer() {  
    newCustomerMenu.click();  
}
```

```
public void clickNewAccount() {
```

```
    WebDriverWait wait =  
        new WebDriverWait(  
            driver,  
            Duration.ofSeconds(10));
```

```
wait.until(  
    ExpectedConditions.elementToBeClickable(  
        newAccountMenu));  
  
JavascriptExecutor js =  
    (JavascriptExecutor) driver;  
  
js.executeScript(  
    "arguments[0].click();",  
    newAccountMenu);  
}
```

```
public void clickFundTransfer() {
```

```
    WebDriverWait wait =  
        new WebDriverWait(  
            driver,  
            Duration.ofSeconds(10));
```

```
wait.until(  
    ExpectedConditions.visibilityOf(  
        fundTransferMenu));
```

```
JavascriptExecutor js =  
    (JavascriptExecutor) driver;
```

```
js.executeScript(  
    "arguments[0].scrollIntoView(true);",  
    fundTransferMenu);
```

```
js.executeScript(  
    "arguments[0].click();",  
    fundTransferMenu);  
}
```

```
public void clickWithdrawal() {  
    withdrawalMenu.click();  
}
```

```
public void clickDeposit() {  
    depositMenu.click();  
}
```

```
public void logout() {
```

```
        logoutMenu.click();
    }
}
```

LoginPage

```
package pages.guru99;
```

```
import base.BasePage;
import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.FindBy;
import utilities.WaitUtils;

public class LoginPage extends BasePage {

    private static final Logger logger =
        LogManager.getLogger(LoginPage.class);

    public LoginPage(WebDriver driver) {
        super(driver);
    }

    @FindBy(name = "uid")
```

```
private WebElement userIdTxt;
```

```
@FindBy(name = "password")
```

```
private WebElement passwordTxt;
```

```
@FindBy(name = "btnLogin")
```

```
private WebElement loginBtn;
```

```
public boolean isLoginPageDisplayed() {
```

```
    logger.info("Verifying Login Page");
```

```
    return userIdTxt.isDisplayed();
```

```
}
```

```
public void enterUserId(String userId) {
```

```
    logger.info("Entering User ID");
```

```
    WaitUtils.waitForVisibility(userIdTxt);
```

```
    userIdTxt.clear();
```

```
    userIdTxt.sendKeys(userId);
```

```
}
```

```
public void enterPassword(String password) {
```

```
    logger.info("Entering Password");
```

```
    WaitUtils.waitForVisibility(passwordTxt);
```

```
    passwordTxt.clear();
```

```
    passwordTxt.sendKeys(password);
```

```
}
```

```
public void clickLogin() {
```

```
    logger.info("Clicking Login Button");
```

```
    WaitUtils.waitForClickable(loginBtn);
```

```
    loginBtn.click();
```

```
}
```

```
public void login(
```

```
    String userId,
```

```
    String password) {
```



```
logger.info("Starting Login Process");
```

```
enterUserId(userId);
```

```
enterPassword(password);
```

```
clickLogin();
```

```
logger.info("Login Process Completed");
```

```
}
```

```
}
```

WithdrawalPage

```
package pages.guru99;
```

```
import base.BasePage;
```

```
import org.apache.logging.log4j.LogManager;
```

```
import org.apache.logging.log4j.Logger;
```

```
import org.openqa.selenium.WebDriver;
```

```
import org.openqa.selenium.WebElement;
```

```
import org.openqa.selenium.support.FindBy;
```

```
import utilities.WaitUtils;
```

```
public class WithdrawalPage extends BasePage {
```

```
private static final Logger logger =  
    LogManager.getLogger(WithdrawalPage.class);
```

```
public WithdrawalPage(WebDriver driver) {  
    super(driver);  
}
```

```
@FindBy(name = "accountno")  
private WebElement accountNumberTxt;
```

```
@FindBy(name = "ammount")  
private WebElement amountTxt;
```

```
@FindBy(name = "desc")  
private WebElement descriptionTxt;
```

```
@FindBy(name = "AccSubmit")  
private WebElement submitBtn;
```

```
public void withdrawAmount(  
    String accountId,  
    String amount,  
    String description) {  
  
    logger.info("Performing Withdrawal");
```

```
accountNumberTxt.clear();  
accountNumberTxt.sendKeys(accountId);
```

```
amountTxt.clear();  
amountTxt.sendKeys(amount);
```

```
descriptionTxt.clear();  
descriptionTxt.sendKeys(description);
```

```
WaitUtils.waitForClickable(submitBtn);
```

```
submitBtn.click();  
}  
}
```

WithdrawalSuccessPage

```
package pages.guru99;
```

```
import base.BasePage;  
import org.openqa.selenium.By;  
import org.openqa.selenium.WebDriver;
```

```
public class WithdrawalSuccessPage extends BasePage {
```

```
public WithdrawalSuccessPage(WebDriver driver) {  
    super(driver);  
}  
  
public boolean isWithdrawalSuccessful() {  
  
    return driver.findElement(  
        By.xpath("//p[contains(text(),'Transaction details of Withdrawal')]"))  
        .isDisplayed();  
}  
  
public String getTransactionId() {  
  
    try {  
  
        return driver.findElement(  
            By.xpath("//td[text()='Transaction ID']/following-sibling::td"))  
            .getText();  
  
    } catch (Exception e) {  
  
        return "NOT AVAILABLE";  
    }  
}
```

```
public void printWithdrawalDetails() {  
  
    System.out.println(  
        "Withdrawal Successful");  
  
    System.out.println(  
        "Transaction ID : "  
        + getTransactionId());  
}  
}
```

Testcases Package

FrameworkValidationTest

```
package testcases;  
  
import base.BaseTest;  
import base.DriverManager;  
import org.testng.Assert;  
import org.testng.annotations.Test;  
  
public class FrameworkValidationTest  
    extends BaseTest {  
  
    @Test  
    public void verifyFrameworkSetup() {
```

```
String title =  
    DriverManager.getDriver()  
        .getTitle();  
  
Assert.assertTrue(  
    title.contains("Guru99"));  
}  
}
```

Testcases.guru99 Package

BankingWorkflowTest

```
package testcases.guru99;  
  
import base.BaseTest;  
import base.DriverManager;  
import org.testng.Assert;  
import org.testng.annotations.Test;  
import pages.guru99.*;  
import utilities.ConfigReader;  
import utilities.RandomDataGenerator;  
import org.testng.annotations.Listeners;  
import listeners.TestListener;  
  
@Listeners(TestListener.class)
```

```
public class BankingWorkflowTest extends BaseTest {

    @Test

    public void completeBankingWorkflow() throws InterruptedException {

        LoginPage loginPage =

            new LoginPage(

                DriverManager.getDriver());

        loginPage.login(

            ConfigReader.getProperty("username"),

            ConfigReader.getProperty("password"));

        HomePage homePage =

            new HomePage(

                DriverManager.getDriver());

        Assert.assertTrue(

            homePage.isManagerHomePageDisplayed());

        // CUSTOMER CREATION

        homePage.clickNewCustomer();

        CustomerPage customerPage =
```

```
new CustomerPage(  
    DriverManager.getDriver());  
  
customerPage.createCustomer(  
    "Saurabh Kumar",  
    "1998-01-01",  
    "Ranchi Main Road",  
    "Ranchi",  
    "Jharkhand",  
    "834001",  
    RandomDataGenerator.generateMobile(),  
    RandomDataGenerator.generateEmail(),  
    "Test@123");  
  
CustomerSuccessPage customerSuccessPage =  
    new CustomerSuccessPage(  
        DriverManager.getDriver());  
  
String customerId =  
    customerSuccessPage.getCustomerId();  
  
System.out.println(  
    "Customer ID = "  
        + customerId);  
  
// ACCOUNT CREATION
```



```
homePage =  
    new HomePage(  
        DriverManager.getDriver());  
  
homePage.clickNewAccount();  
  
AccountPage accountPage =  
    new AccountPage(  
        DriverManager.getDriver());  
  
accountPage.createAccount(  
    customerId,  
    "Savings",  
    "5000");  
  
AccountSuccessPage accountSuccessPage =  
    new AccountSuccessPage(  
        DriverManager.getDriver());  
  
String account1 =  
    accountSuccessPage.getAccountId();  
  
System.out.println(  
    "Account 1 = "  
    + account1);
```

```
homePage =  
    new HomePage(  
        DriverManager.getDriver());  
  
homePage.clickNewAccount();  
  
accountPage =  
    new AccountPage(  
        DriverManager.getDriver());  
  
accountPage.createAccount(  
    customerId,  
    "Savings",  
    "3000");  
  
accountSuccessPage =  
    new AccountSuccessPage(  
        DriverManager.getDriver());  
  
String account2 =  
    accountSuccessPage.getAccountId();  
  
System.out.println(  
    "Account 2 = " + account2);  
  
homePage =
```

```
        new HomePage(  
            DriverManager.getDriver());  
  
homePage.clickNewAccount();  
  
accountPage =  
    new AccountPage(  
        DriverManager.getDriver());  
  
accountPage.createAccount(  
    customerId,  
    "Savings",  
    "3000");  
  
accountSuccessPage =  
    new AccountSuccessPage(  
        DriverManager.getDriver());  
  
String account2f =  
    accountSuccessPage.getAccountId();  
  
System.out.println(  
    "Account 2 = " + account2);  
  
homePage =  
    new HomePage(  
        DriverManager.getDriver());
```

```
DriverManager.getDriver());
```

```
homePage.clickFundTransfer();
```

```
FundTransferPage fundTransferPage =
```

```
    new FundTransferPage(  
        DriverManager.getDriver());
```

```
fundTransferPage.transferFund(  
    account1,
```

```
    account2,
```

```
    "500",
```

```
    "Transfer");
```

```
FundTransferSuccessPage fundTransferSuccessPage =
```

```
    new FundTransferSuccessPage(  
        DriverManager.getDriver());
```

```
Assert.assertTrue(  
    fundTransferSuccessPage
```

```
        .isFundTransferSuccessful());
```

```
fundTransferSuccessPage
```

```
    .printFundTransferDetails();
```

```
System.out.println(  
    "Fund Transfer Completed");
```

```
homePage =  
    new HomePage(  
        DriverManager.getDriver());  
  
homePage.clickWithdrawal();  
  
WithdrawalPage withdrawalPage =  
    new WithdrawalPage(  
        DriverManager.getDriver());  
  
withdrawalPage.withdrawAmount(  
    account1,  
    "100",  
    "ATM");  
  
WithdrawalSuccessPage withdrawalSuccessPage =  
    new WithdrawalSuccessPage(  
        DriverManager.getDriver());  
  
Assert.assertTrue(  
    withdrawalSuccessPage.isWithdrawalSuccessful());
```

```
withdrawalSuccessPage.printWithdrawalDetails();
```

```
try {
```

```
    homePage.clickDeposit();
```

```
    DepositPage depositPage =
```

```
        new DepositPage(  
            DriverManager.getDriver());
```

```
    depositPage.depositAmount(  
        account1,
```

```
        "1000",
```

```
        "Cash Deposit");
```

```
    Thread.sleep(3000);
```

```
    } catch (Exception e) {
```

```
        System.out.println(  
            "Deposit Failed : "
```

```
            + e.getMessage());
```

```
        }
```

```
System.out.println(
    "Navigating back to Home Page");

DriverManager.getDriver().navigate().back();

Thread.sleep(2000);

DriverManager.getDriver().get(
    "https://demo.guru99.com/V4/manager/Logout.php");

try {

    DriverManager.getDriver()
        .switchTo()
        .alert()
        .accept();

    System.out.println(
        "Logout Successful");

} catch (Exception e) {

    System.out.println(
        "Logout Alert Not Found");

}
}
```

```
}
```

LoginTest

```
package testcases.guru99;
```

```
import base.BaseTest;
```

```
import base.DriverManager;
```

```
import org.testng.Assert;
```

```
import org.testng.annotations.Test;
```

```
import pages.guru99.HomePage;
```

```
import pages.guru99.LoginPage;
```

```
import utilities.ConfigReader;
```

```
import java.time.Duration;
```

```
import org.openqa.selenium.Alert;
```

```
import org.openqa.selenium.support.ui.ExpectedConditions;
```

```
import org.openqa.selenium.support.ui.WebDriverWait;
```

```
public class LoginTest extends BaseTest {
```

```
    @Test
```

```
    public void verifyValidLogin() {
```

```
        LoginPage loginPage =
```

```
            new LoginPage(
```

```
                DriverManager.getDriver());
```



```
loginPage.login(  
    ConfigReader.getProperty("username"),  
    ConfigReader.getProperty("password"));  
  
HomePage homePage =  
    new HomePage(  
        DriverManager.getDriver());  
  
Assert.assertTrue(  
    homePage.isManagerHomePageDisplayed(),  
    "Login Failed");  
}
```

@Test

```
public void verifyInvalidLogin() {
```

```
    LoginPage loginPage =  
        new LoginPage(  
            DriverManager.getDriver());
```

```
    loginPage.login(  
        "invalidUser",  
        "invalidPassword");
```

```
    WebDriverWait wait =  
        new WebDriverWait(  
            driver, 10, true);
```

```
DriverManager.getDriver(),  
Duration.ofSeconds(10));
```

```
Alert alert =  
    wait.until(  
        ExpectedConditions.alertIsPresent());
```

```
String alertText =  
    alert.getText();
```

```
System.out.println(  
    "Alert Message = "  
        + alertText);
```

```
Assert.assertTrue(  
    alertText.contains("not valid"),  
    "Invalid login message not displayed");
```

```
    alert.accept();  
}
```

```
@Test  
public void verifyEmptyLogin() {
```

```
LoginPage loginPage =  
    new LoginPage(  
        DriverManager.getDriver());  
  
loginPage.login(  
    "",  
    "");  
  
WebDriverWait wait =  
    new WebDriverWait(  
        DriverManager.getDriver(),  
        Duration.ofSeconds(10));  
  
Alert alert =  
    wait.until(  
        ExpectedConditions.alertIsPresent());  
  
String alertText =  
    alert.getText();  
  
System.out.println(  
    "Alert Message = "  
        + alertText);  
  
Assert.assertTrue(  
    alertText.length() > 0,
```

```
"Expected alert message not displayed");
```

```
    alert.accept();
```

```
    }
```

```
}
```

CustomerTest

```
package testcases.guru99;
```

```
import base.BaseTest;
```

```
import base.DriverManager;
```

```
import org.testng.Assert;
```

```
import org.testng.annotations.Test;
```

```
import pages.guru99.CustomerPage;
```

```
import pages.guru99.CustomerSuccessPage;
```

```
import pages.guru99.HomePage;
```

```
import pages.guru99.LoginPage;
```

```
import utilities.ConfigReader;
```

```
import utilities.ExcelUtils;
```

```
import utilities.RandomDataGenerator;
```

```
import utilities.TestContext;
```

```
import utilities.TestDataProvider;
```

```
public class CustomerTest extends BaseTest {
```

```
    @Test(
```

```
        dataProvider = "customerData",
        dataProviderClass =
            TestDataProvider.class)
public void verifyCustomerCreation(

    String customerName,
    String dob,
    String address,
    String city,
    String state,
    String pin,
    String password) {

LoginPage loginPage =
    new LoginPage(
        DriverManager.getDriver());

loginPage.login(
    ConfigReader.getProperty("username"),
    ConfigReader.getProperty("password"));

HomePage homePage =
    new HomePage(
        DriverManager.getDriver());

homePage.clickNewCustomer();
```

```
CustomerPage customerPage =  
    new CustomerPage(  
        DriverManager.getDriver());  
  
customerPage.createCustomer(  
    customerName,  
    dob,  
    address,  
    city,  
    state,  
    pin,  
    RandomDataGenerator.generateMobile(),  
    RandomDataGenerator.generateEmail(),  
    password);  
  
CustomerSuccessPage successPage =  
    new CustomerSuccessPage(  
        DriverManager.getDriver());  
  
String customerId =  
    successPage.getCustomerId();  
  
Assert.assertNotNull(customerId);  
  
System.out.println(
```

```
        "Customer Created : "  
        + customerId);  
    }  
}
```

AccountTest

```
package testcases.guru99;  
  
import base.BaseTest;  
import base.DriverManager;  
import org.testng.Assert;  
import org.testng.annotations.Test;  
import pages.guru99.*;  
import utilities.ConfigReader;  
import utilities.RandomDataGenerator;  
import utilities.TestContext;  
  
public class AccountTest extends BaseTest {  
  
    @Test  
    public void verifyAccountCreation() {  
  
        LoginPage loginPage =  
            new LoginPage(  
                DriverManager.getDriver());
```

```
loginPage.login(  
    ConfigReader.getProperty("username"),  
    ConfigReader.getProperty("password"));  
  
HomePage homePage =  
    new HomePage(  
        DriverManager.getDriver());  
  
homePage.clickNewCustomer();  
  
CustomerPage customerPage =  
    new CustomerPage(  
        DriverManager.getDriver());  
  
customerPage.createCustomer(  
    "Saurabh Kumar",  
    "1998-01-01",  
    "Ranchi Main Road",  
    "Ranchi",  
    "Jharkhand",  
    "834001",  
    RandomDataGenerator.generateMobile(),  
    RandomDataGenerator.generateEmail(),  
    "Test@123");  
  
CustomerSuccessPage customerSuccessPage =
```



```
new CustomerSuccessPage(  
    DriverManager.getDriver());
```

```
String customerId =  
    customerSuccessPage.getCustomerId();
```

```
System.out.println(  
    "Customer ID From Context = "  
    + TestContext.getCustomerId());
```

```
homePage =  
    new HomePage(  
        DriverManager.getDriver());
```

```
homePage.clickNewAccount();
```

```
AccountPage accountPage =  
    new AccountPage(  
        DriverManager.getDriver());
```

```
accountPage.createAccount(  
    customerId,  
    "Savings",  
    "5000");
```

```
AccountSuccessPage accountSuccessPage =  
    new AccountSuccessPage(  
        DriverManager.getDriver());  
  
String accountId =  
    accountSuccessPage.getAccountId();  
  
Assert.assertNotNull(accountId);  
  
System.out.println(  
    "Generated Account ID : "  
        + accountId);  
  
System.out.println(  
    "Stored Account ID : "  
        + TestContext.getAccountId());  
}  
}
```

DepositTest

```
package testcases.guru99;  
  
import org.testng.Assert;  
import org.testng.annotations.Test;  
  
import base.BaseTest;
```

```
public class DepositTest extends BaseTest {
```

```
    @Test
```

```
    public void verifyDeposit() {
```

```
        // Login
```

```
        // Create Customer
```

```
        // Create Account
```

```
        // Deposit Amount
```

```
        Assert.assertTrue(true);
```

```
    }
```

```
}
```

Utilities Package

ConfigReader

```
package utilities;
```

```
import java.io.FileInputStream;
```

```
import java.util.Properties;
```

```
public class ConfigReader {

    private static final Properties properties =
        new Properties();

    static {

        try {

            FileInputStream fis =
                new FileInputStream(
                    "src/test/resources/config/config.properties");

            properties.load(fis);

        } catch (Exception e) {

            throw new RuntimeException(e);
        }
    }

    public static String getProperty(String key) {

        return properties.getProperty(key);
    }
}
```

RandomDataGenerator

```
package utilities;
```

```
import java.util.UUID;
```

```
public class RandomDataGenerator {
```

```
    public static String generateEmail() {
```

```
        return "user"
```

```
            + UUID.randomUUID()
```

```
            .toString()
```

```
            .substring(0,6)
```

```
            + "@gmail.com";
```

```
    }
```

```
    public static String generateMobile() {
```

```
        String timestamp =
```

```
            String.valueOf(System.currentTimeMillis());
```

```
        return "98"
```

```
            + timestamp.substring(
```

```
                timestamp.length()-8);
```

```
    }
```

```
}
```

ReportManager

```
package utilities;

import com.aventstack.extentreports.*;
import com.aventstack.extentreports.reporter.ExtentSparkReporter;

public class ReportManager {

    private static ExtentReports extent;

    public static ExtentReports getReportInstance() {

        if (extent == null) {

            System.out.println("Extent Report Initialized");

            ExtentSparkReporter spark =
                new ExtentSparkReporter(
                    "reports/ExtentReport.html");

            spark.config().setReportName(
                "Guru99 Banking Automation Framework");

            spark.config().setDocumentTitle(
                "Banking Automation Report");
```

```
extent = new ExtentReports();
```

```
extent.attachReporter(spark);
```

```
// Project Details
```

```
extent.setSystemInfo(  
    "Project Name",  
    "Banking Automation Framework");
```

```
extent.setSystemInfo(  
    "Tester",  
    "Saurabh Kumar");
```

```
extent.setSystemInfo(  
    "Browser",  
    ConfigReader.getProperty("browser"));
```

```
extent.setSystemInfo(  
    "Environment",  
    "QA");
```

```
extent.setSystemInfo(  
    "Framework",  
    "Selenium + TestNG + Maven");
```

```
extent.setSystemInfo(  
    "Platform",  
    "Windows");
```

```
        "CI/CD",  
        "Jenkins");  
  
    extent.setSystemInfo(  
        "Containerization",  
        "Docker");  
  
    extent.setSystemInfo(  
        "OS",  
        System.getProperty("os.name"));  
  
    extent.setSystemInfo(  
        "Java Version",  
        System.getProperty("java.version"));  
    }  
  
    return extent;  
}  
}
```

ScreenshotUtils

```
package utilities;
```

```
import base.DriverManager;  
import org.apache.commons.io.FileUtils;  
import org.openqa.selenium.*;
```



```
import java.io.File;

import java.text.SimpleDateFormat;

import java.util.Date;


public class ScreenshotUtils {


    public static String capture(String testName) {


        try {


            File source =

                ((TakesScreenshot)

                    DriverManager.getDriver())

                    .getScreenshotAs(OutputType.FILE);


            String timestamp =

                new SimpleDateFormat(

                    "yyyyMMddHHmmss")

                    .format(new Date());


            String folderPath = "screenshots";


            File folder = new File(folderPath);


            if (!folder.exists()) {
```

```
        folder.mkdirs();  
    }  
}
```

```
String destination =  
    new File(  
        folderPath,  
        testName + "_" +  
            timestamp  
            + ".png")  
        .getAbsolutePath();
```

```
FileUtils.copyFile(  
    source,  
    new File(destination));
```

```
System.out.println(  
    "Screenshot Saved At : "  
    + destination);
```

```
return destination;
```

```
} catch (Exception e) {  
  
    e.printStackTrace();
```

```
        //throw new RuntimeException(e);
    }

    return testName;
}
}
```

TextContext

package utilities;

```
public class TextContext {

    private static ThreadLocal<String> customerId =
        new ThreadLocal<>();

    private static ThreadLocal<String> accountId =
        new ThreadLocal<>();

    public static String getCustomerId() {
        return customerId.get();
    }

    public static void setCustomerId(String id) {
        customerId.set(id);
    }
}
```

```
public static String getAccountId() {  
    return accountId.get();  
}  
  
public static void setAccountId(String id) {  
    accountId.set(id);  
}  
}
```

ExcelUtils

```
package utilities;  
  
import org.apache.poi.ss.usermodel.*;  
import org.apache.poi.xssf.usermodel.XSSFWorkbook;  
  
import java.io.FileInputStream;  
  
public class ExcelUtils {  
  
    private static Workbook workbook;  
  
    static {  
  
        try {
```

```

        FileInputStream fis =
            new FileInputStream(
                "src/test/resources/testdata/TestData.xlsx");

        workbook =
            new XSSFWorkbook(fis);

    } catch (Exception e) {

        throw new RuntimeException(e);
    }
}

public static String getCellData(
    String sheetName,
    int row,
    int column) {

    Sheet sheet =
        workbook.getSheet(sheetName);

    return sheet.getRow(row)
        .getCell(column)
        .toString();
}

```

```
public static int getRowCount(  
    String sheetName) {  
  
    return workbook.getSheet(sheetName)  
        .getLastRowNum();  
}  
  
}
```

WaitUtils

```
package utilities;

import base.DriverManager;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.ui.*;

import java.time.Duration;

public class WaitUtils {

    private static final int TIMEOUT = 15;

    public static void waitForVisibility(
        WebElement element) {

        WebDriverWait wait =
            new WebDriverWait(
```

```

        DriverManager.getDriver(),
        Duration.ofSeconds(TIMEOUT));

    wait.until(
        ExpectedConditions.visibilityOf(element));
}

public static void waitForClickable(
    WebElement element) {

    WebDriverWait wait =
        new WebDriverWait(
            DriverManager.getDriver(),
            Duration.ofSeconds(TIMEOUT));

    wait.until(
        ExpectedConditions.elementToBeClickable(element));
}
}

```

Log4j2.xml

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<Configuration status="WARN">
```

<Appenders>

<Console name="Console">

<PatternLayout

pattern="%d [%t] %-5level %logger - %msg%n"/>

</Console>

<File name="File"

fileName="logs/automation.log">

<PatternLayout

pattern="%d %-5level %logger - %msg%n"/>

</File>

</Appenders>

<Loggers>

<Root level="info">

<AppenderRef ref="Console"/>

<AppenderRef ref="File"/>

</Root>

</Loggers>

</Configuration>

Pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
```

```
<modelVersion>4.0.0</modelVersion>
```

```
<groupId>BankingAutomationFramework</groupId>
```

```
<artifactId>BankingAutomationFramework</artifactId>
```

```
<version>0.0.1-SNAPSHOT</version>
```

```
<properties>
```

```
<maven.compiler.source>21</maven.compiler.source>
```

```
<maven.compiler.target>21</maven.compiler.target>
```

```
<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
```

```
</properties>
```

```
<dependencies>
```

```
<!-- Selenium -->
```

```
<dependency>
```

```
<groupId>org.seleniumhq.selenium</groupId>
```

```
<artifactId>selenium-java</artifactId>

<version>4.34.0</version>

</dependency>

<!-- TestNG -->

<dependency>

    <groupId>org.testng</groupId>

    <artifactId>testng</artifactId>

    <version>7.11.0</version>

</dependency>

<!-- WebDriverManager -->

<dependency>

    <groupId>io.github.bonigarcia</groupId>

    <artifactId>webdrivermanager</artifactId>

    <version>6.1.0</version>

</dependency>

<!-- Apache POI -->

<dependency>

    <groupId>org.apache.poi</groupId>

    <artifactId>poi-ooxml</artifactId>

    <version>5.4.1</version>

</dependency>

<!-- Extent Report -->
```

```
<dependency>
  <groupId>com.aventstack</groupId>
  <artifactId>extentreports</artifactId>
  <version>5.1.2</version>
</dependency>
```

```
<!-- Log4j -->
<dependency>
  <groupId>org.apache.logging.log4j</groupId>
  <artifactId>log4j-core</artifactId>
  <version>2.25.1</version>
</dependency>
```

```
<dependency>
  <groupId>org.apache.logging.log4j</groupId>
  <artifactId>log4j-api</artifactId>
  <version>2.25.1</version>
</dependency>
```

```
<!-- Commons IO -->
<dependency>
  <groupId>commons-io</groupId>
  <artifactId>commons-io</artifactId>
  <version>2.18.0</version>
</dependency>
```

</dependencies>

<build>

<plugins>

<plugin>

<groupId>org.apache.maven.plugins</groupId>

<artifactId>maven-surefire-plugin</artifactId>

<version>3.2.5</version>

<configuration>

<suiteXmlFiles>

<suiteXmlFile>testng.xml</suiteXmlFile>

</suiteXmlFiles>

</configuration>

</plugin>

</plugins>

</build>

</project>

Testing.xml

```
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
```

```
<suite name="BankingSuite"
```

```
  parallel="tests"
```

```
  thread-count="3">
```

```
  <listeners>
```

```
    <listener class-name="listeners.TestListener"/>
```

```
  </listeners>
```

```
    <test name="LoginTest">
```

```
      <classes>
```

```
        <class name="testcases.guru99.LoginTest"/>
```

```
      </classes>
```

```
    </test>
```

```
    <test name="CustomerTest">
```

```
      <classes>
```

```
        <class name="testcases.guru99.CustomerTest"/>
```

```
      </classes>
```

```
    </test>
```

```
<test name="BankingWorkflowTest">
```

```
<classes>
```

```
<class name="testcases.guru99.BankingWorkflowTest"/>
```

```
</classes>
```

```
</test>
```

```
<test name="AccountTest">
```

```
<classes>
```

```
<class name="testcases.guru99.AccountTest"/>
```

```
</classes>
```

```
</test>
```

```
</suite>
```

Jenkinsfile

```
pipeline {
```

```
    agent any
```

```
    stages {
```

```
        stage('Build & Test') {
```

```
            steps {
```

```
    dir('BankingAutomationFramework') {  
        bat 'mvn clean test'  
    }  
}  
}
```

```
post {  
    always {  
        archiveArtifacts artifacts: 'BankingAutomationFramework/reports/*.html',  
            allowEmptyArchive: true  
  
        archiveArtifacts artifacts: 'BankingAutomationFramework/screenshots/*.png',  
            allowEmptyArchive: true  
    }  
}
```

Dockerfile

```
FROM maven:3.9.11-eclipse-temurin-21
```

```
RUN apt-get update && apt-get install -y \
```

```
    wget \
```

```
    gnupg \
```

```
    unzip \
```

```
&& wget -q -O - https://dl.google.com/linux/linux_signing_key.pub | gpg --dearmor -o /usr/share/keyrings/google.gpg \

&& echo "deb [signed-by=/usr/share/keyrings/google.gpg]
http://dl.google.com/linux/chrome/deb/ stable main" > /etc/apt/sources.list.d/google.list \

&& apt-get update \

&& apt-get install -y google-chrome-stable
```

WORKDIR /app

COPY ..

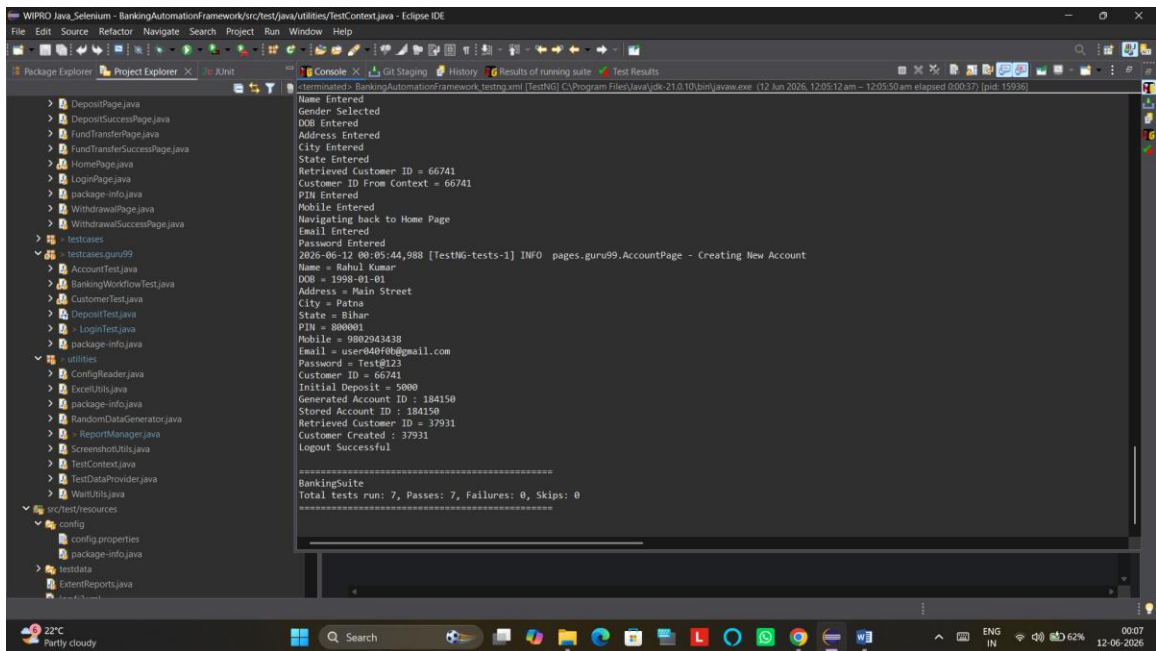
RUN mvn clean install -DskipTests

CMD ["mvn","test"]

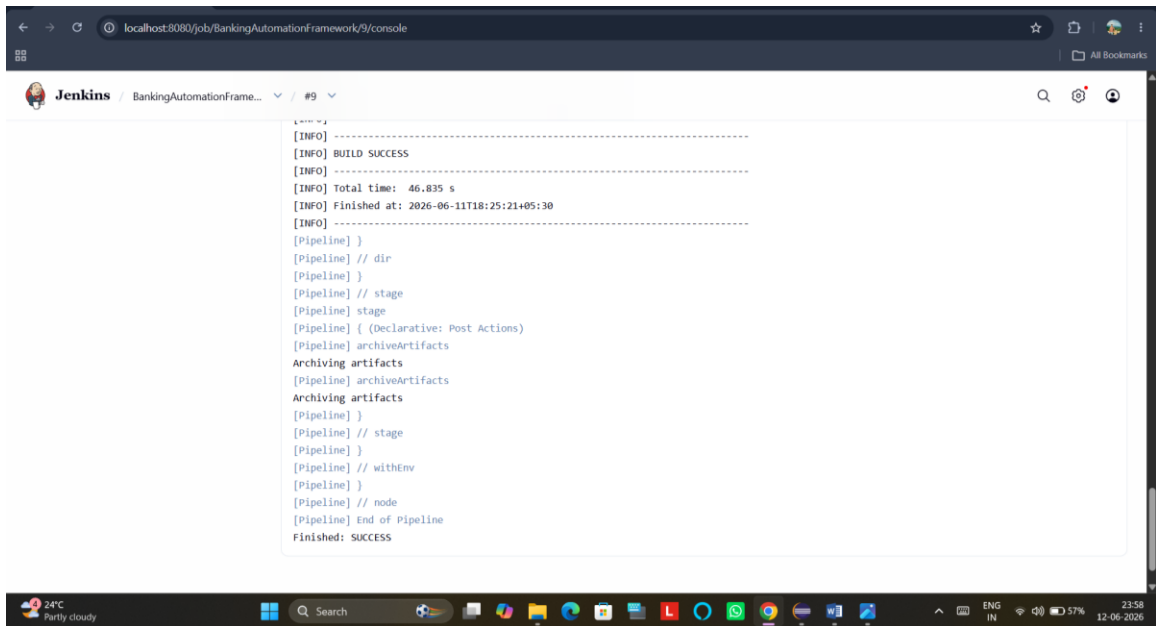
TestData.xlsx (Excel File)

Name	DOB	Address	City	State	Pin	Password
Saurabh Kumar	01-01-1998	Ranchi Main Road	Ranchi	Jharkhand	834001	Test@123
Rahul Kumar	10-05-1995	Main Street	Patna	Bihar	800001	Test@123

Eclipse Console Output



Jenkins Outputs



Jenkins / BankingAutomationFramework

Status: ✔ **BankingAutomationFramework**

Capstone Project

Last Successful Artifacts:

Artifact Name	Size	Action
ExtentReport.html	7.57 KB	view
completeBankingWorkflow_20260610161032.png	16.86 KB	view
completeBankingWorkflow_20260610165138.png	16.86 KB	view
completeBankingWorkflow_20260610165559.png	16.86 KB	view
completeBankingWorkflow_20260610213738.png	47.17 KB	view
completeBankingWorkflow_20260610213831.png	47.60 KB	view
completeBankingWorkflow_20260610213941.png	11.78 KB	view
completeBankingWorkflow_20260611182218.png	10.61 KB	view
verifyAccountCreation_20260608153056.png	16.86 KB	view
verifyCustomerCreation_20260610185719.png	10.63 KB	view
verifyCustomerCreation_20260610192215.png	16.86 KB	view
verifyCustomerCreation_20260610213837.png	11.78 KB	view
verifyCustomerCreation_20260610214710.png	16.86 KB	view

Permalinks

- Last build (#9), 1 day 5 hr ago
- Last stable build (#9), 1 day 5 hr ago
- Last successful build (#9), 1 day 5 hr ago
- Last failed build (#8), 1 day 5 hr ago
- Last unsuccessful build (#8), 1 day 5 hr ago
- Last completed build (#9), 1 day 5 hr ago

Jenkins / BankingAutomationFramework / #9 / Stages

✔ < #9

Started by Saurabh Kumar | Started 6 hr 7 min ago | Queued 12 ms | Took 55 sec | Artifacts

Start → Checkout SCM → Build & Test → Post Actions → End

Post Actions (0.21s, Started 6h 6m ago)

- Archive the artifacts (72ms)
- Archive the artifacts (83ms)

Archiving artifacts

Git Build Data

Revision: abbc6a64c0d0a9cf25eebfcd35bc09acae3156b0

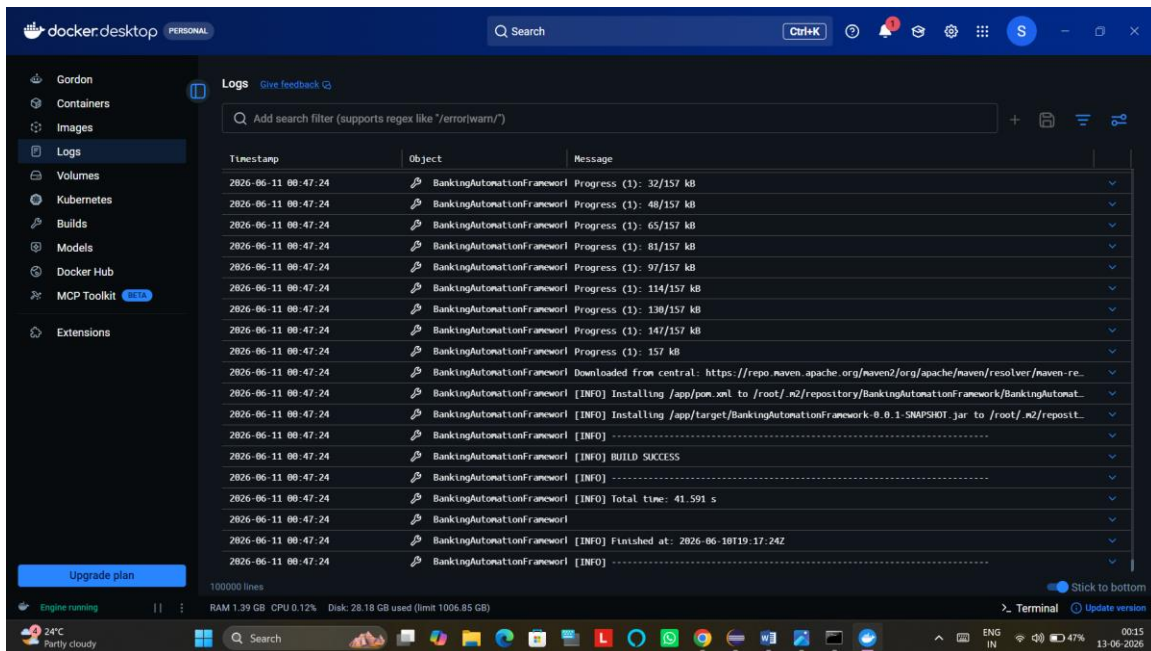
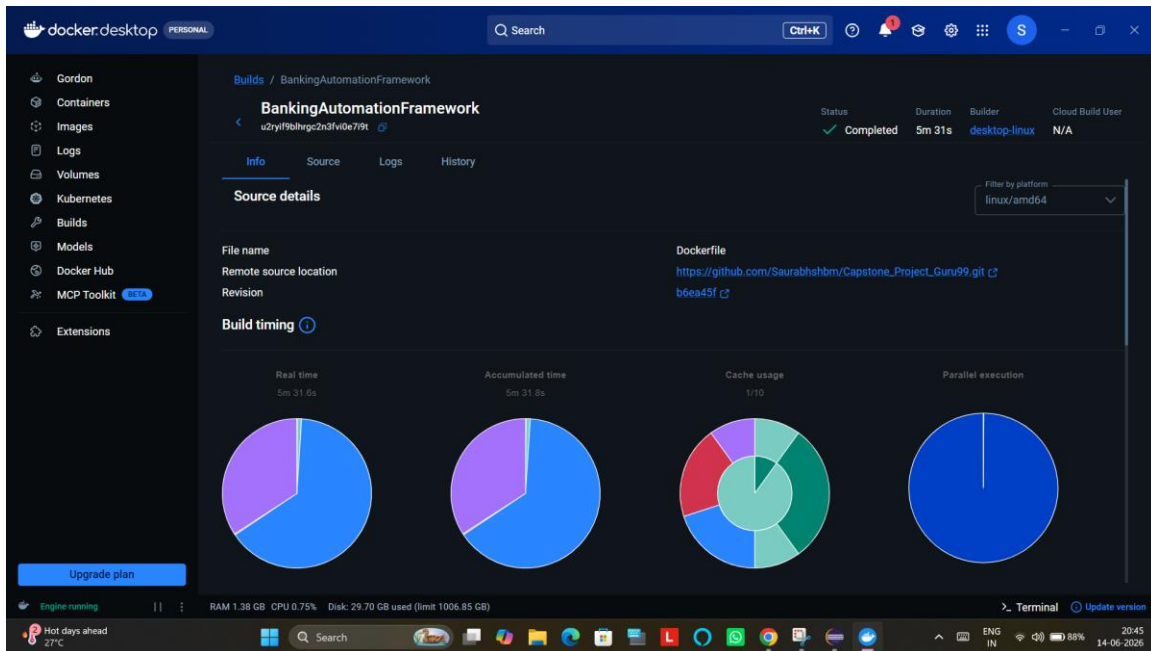
Repository: https://github.com/Saurabhshbm/Capstone_Project_Guru99.git

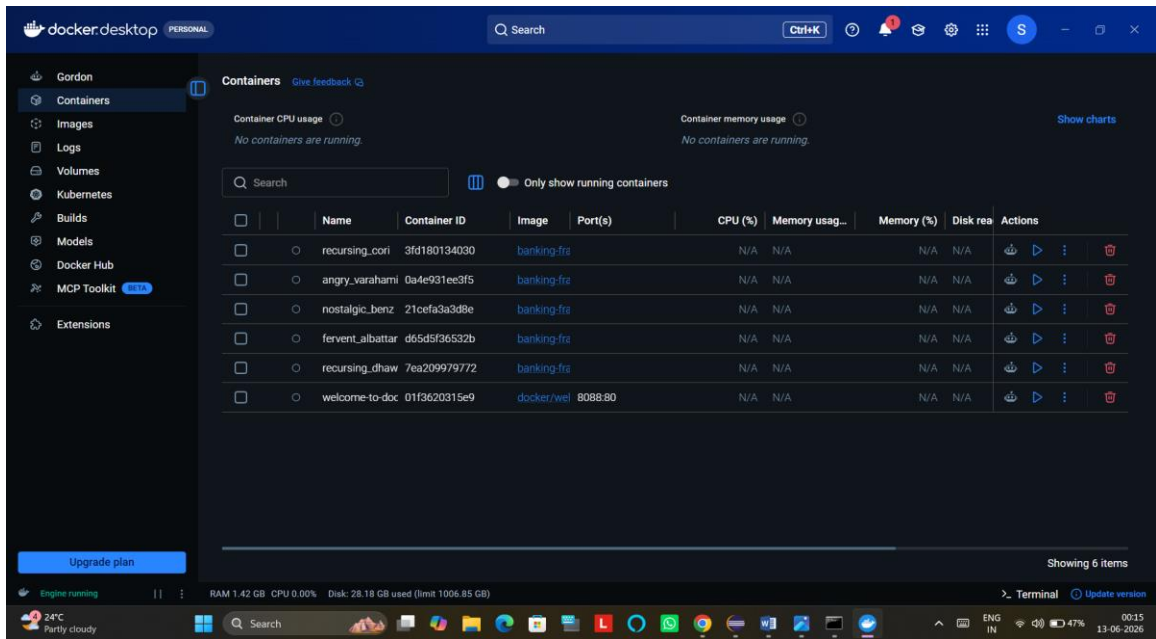
- refs/remotes/origin/master

Built Branches

- refs/remotes/origin/master: Build #9 of Revision
abbcb6a64c0d0a9cf25eebfcd35bc09acae3156b0 (refs/remotes/origin/master)

Docker Output:-





```
Command Prompt

C:\Users\saura\git\Capstone_project\BankingAutomationFramework>docker build --no-cache -t banking-framework .
[+] Building 331.9s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 595B
=> [internal] load metadata for docker.io/library/maven:3.9.11-eclipse-temurin-21
=> [auth] library/maven:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> CACHED [1/5] FROM docker.io/library/maven:3.9.11-eclipse-temurin-21@sha256:6fdc855a6ed81d288ca7ca37ac6ff5e930
=> => resolve docker.io/library/maven:3.9.11-eclipse-temurin-21@sha256:6fdc855a6ed81d288ca7ca37ac6ff5e9308b61248
=> [internal] load build context
=> => transferring context: 940.94kB
=> [2/5] RUN apt-get update && apt-get install -y wget gnupg unzip && wget -q -O - https://dl.
=> [3/5] WORKDIR /app
=> [4/5] COPY . .
=> [5/5] RUN mvn clean install -DskipTests
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:09dee6e9ec8501c4106300b85be4c08d4fac3159fa76d3c1879f25f9e2982507
=> => exporting config sha256:7541aff7293a31b493e8c61fb3abef63f41bafce387973d8ca135d19893abb4
=> => exporting attestation manifest sha256:053cd58d342b8b1e36e146dff78d18c05b2439285e631624cd07511950cfc90
=> => exporting manifest list sha256:cd2ec29b390510aeb8b9dac3ffe6dc18501ac9384218bef6b68ae596687163c
=> => naming to docker.io/library/banking-framework:latest
=> => unpacking to docker.io/library/banking-framework:latest

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/u2ryif9b1hrgc2n3fvi0e7i9t

C:\Users\saura\git\Capstone_project\BankingAutomationFramework>
```

inside the container (if configured in your Dockerfile).

